

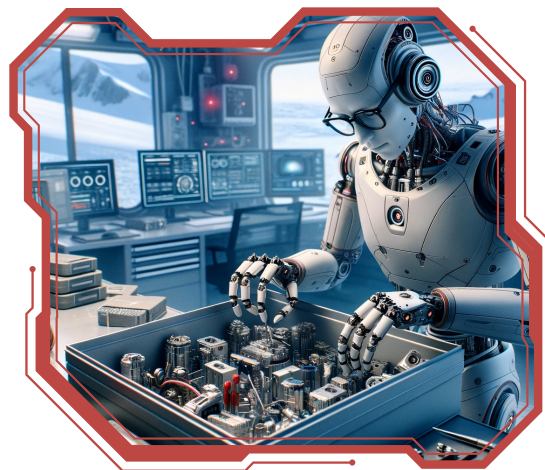
SESSION 3: CONSISTENT PROJECT STRUCTURE

File organization, overlap of git repositories, Rstudio projects and workflows

Kosmas Hench

2025-05-26

*“Reproducibility starts in the planning stage, with sound data management practices. It does not arise simply from sharing data and code online after a project is done. It is difficult to reproduce research when data are disorganized or missing, or when it is impossible to determine where or how data originated” (Alston and Rick, 2021). As this quote stresses, transparency alone is not enough to ensure other people can follow and understand your analysis code. Even with minimally complex projects, dumping all your files in an unstructured way can be a very effective way to **prevent** others from understanding what you did: obfuscation by information overload is a real thing. Documentation of your code is one tool to combat confusion (Session 1) – predictability is another one. In this session we are going to cover how a consistent structure can help you and others to not get lost in your project. Besides, many of the tools discussed in this workshop work best when applying a project based design.*



created using dale3

1. PROJECT BASED THINKING

This session is about the structure and efficient use of projects, however I choose to remain somewhat indifferent what exactly constitutes a project. Basically, a project can be what you choose it to be: it is a particular modular piece of analysis or a task that you are working on. In many cases, a project will be equated with a study/publication. However, this is not necessarily so – depending on the circumstances, a study might be consisting of multiple projects (e.g. one per contributing co-author), or a single project might be the basis for multiple studies that all depend on shared preparations (e.g. processing of shared raw data).

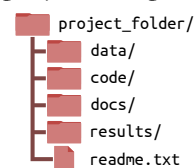
For the purpose of this workshop, you can think of a project as complete logical unit of a scientific investigation that typically includes data, computing code and that produces new data as a result. It can include the analysis and the interpretation of the results (e.g. a manuscript for a publication), but for our purposes it does not have to include those.

In practice, a project translates to a particular folder on your computer. It turns out many people have wondered about effective ways to structure projects, and there the recommendations seem to converge around a similar design principle (Noble, 2009; Wilson *et al.*, 2017; Figueiredo *et al.*, 2022).

The key points here are that project should be self-contained (that is: it should contain all analyzed data and code), and it's structure mainly separates the input, instructions for data processing and produced results. Usually, you also want to include a plain text readme file (.txt or .md) as a sort of landing page for your project, where you provide the most basic information related to the project. For more complex projects, a folder dedicated to documentation might be needed and if you include the manuscript in the project, that also might go into its own subdirectory.

Project: A single folder on your computer containing a closed logical unit of a scientific investigation, including data, code and results.

Gist of a recommended project setup:



2. REASONING BEHIND THE PROJECT STRUCTURE

The proposed organization of projects comes with several benefits:

Orientation. Firstly, it is generic enough to be generally applicable and to provide a sense of orientation even for an audience that is completely naive to the project (or even to field of study or type of analysis conducted). For any kind of more detailed explanation of your setup, make use of that `readme` file – in most cases this will be the first point of contact with your project folder, so a gentle overview of the repository content is well-placed here. Similarly to code documentation, striking the right balance between under- and over-reporting might be somewhat tricky as you want to educate the reader enough to not get lost in your code without overloading them. Helpful content for a publication related code repository includes for example a list of which exact scripts produce which figures and tables, or an overview in which scripts the main analyses are run. Another typical element of a `readme` might be references to a linked publication or data-repositories. If you feel like in-depth explanations are required to be able to really follow your work, you can point to your `docs/` folder, where you can include more elaborate documentation that goes into the intricate details of your analysis.

All is One. Secondly, this overall structure allows for different reproducibility tools to neatly ‘click into place’ and work together. This is because most of these tools are designed to manage a particular aspect of a focal directory, such as for example the file history (`git`) or the execution order of the analysis steps (`snakemake`). In a way, these are different perspectives on the same subject (the project). Thus, depending on our focal topic, we might be talking about for example our `git` repository, our `Rstudio` project, or our `snakemake` pipeline. Yet, in practice, all of these refer to the same directory: your project folder.

Modularity. Furthermore, this structure also allows to cleanly separate your raw data, code and results and deposit each of them in their own specialized repositories (remember SESSION 1). Conversely, with respect to reproducibility, it also allows easily to re-assemble the original project folder from the separate chunks. Ideally, it should be possible to rerun your analysis by downloading the main scaffolding structure including the code of your project in the form of a `git` repository from zenodo (`readme`, `code/`, `docs/`), plugging in the raw data e.g. from PANGAEA (`data/`) and recreating the results reported in your manuscript and deposited at e.g. Dryad (`results/`).

Mobility. Designing the projects as a self-sufficient folder also has the potential to affect the coding style of the analysis. If all input and output files are living within a single directory, one can use paths *relative* to the main project folder instead of the *absolute paths* of these files. This makes it much easier to transplant the project to a different computer and run it there without having to adjust a lot of code (very likely, the person that benefits most from this will be you...). Technically, this project structure makes it quite obvious what the appropriate **working directory** for running your code should be: the project folder itself.

3. REALITY CHECK: USE LINKS TO RE-USE DATA

The idea of keeping everything needed for the analysis contained within the project folder can lead to one issue when you have several projects relying on the same input data: you might be forced to create multiple copies of the same data (with each copy living in a different project folder).

This is not ideal for several reasons: Depending on what field you are working in, your

Absolute path:
`/folder/existing/only`
`/on/my/computer/`
(note the leading `s/`)

Relative path:
`./sub-folder/of/this/`
`directory/`
(note the leading `./`)

raw data might consist of many and/or large input files (`#big_data`), and obviously, copying these into multiple places would simply be a waste of disc space. Beyond just being uneconomical, parallel versions of the same data create ambiguity when version conflicts arise. If a file is for some reason updated in one project, but not in another, it might later be impossible to tell which file represents the ‘true’ version and which one is outdated.

Despite these issues, it really is advisable to structure all projects such that all input files are located within them, and that the used code does not point to files located elsewhere. This is because the mobility/self-sufficiency aspect is quite an important feature to preserve for reproducibility. One elegant way to resolve this conflict is to keep the real version of the data that you need to re-use outside the project, but then, inside the `project_folder/data/` directory to create **symbolic links** pointing to the true files (instead of copying them). That way it will always be clear which file is the real deal, and if you change this one, the data will be updated in all projects that refer to it. At the same time within the projects, the code can still be written assuming the input files are located within the project’s sphere of influence. And as symbolic links are negligible with respect to file size, that way you can create as many ‘copies’ of the data as you wish without any additional disk space footprint.

4. R SPECIFIC BENEFITS: RSTUDIO PROJECTS

Code Management. When using R as part of your analysis, you can benefit from additional features by turning your folder into a dedicated RStudio Project (Bryan *et al.* 2024, Chapter 3: Project-oriented workflow). All that is required to turn a directory into an RStudio Project is the creation of a small config file (`project_name.Rproj`) in the main project folder, which can be easily done within RStudio. Particularly when using RStudio as IDE, working with RStudio projects allows to nicely bundle all project related analysis and conveniently navigate between different projects within the IDE.

Working Directory. Yet, there are advantages to RStudio projects beyond mere organization of R scripts, such as the explicit declaration of the intended working directory for the code contained within your project (which is the folder containing the `.Rproj` file – your project folder). Basically, the usage of RStudio projects resolves the need to specify the working directory using `setwd()`. This is a good thing, because `setwd()` typically points to a path that only exists on the computer that was used when the script was originally written. In contrast to this, the `.Rproj` can be copied alongside the rest of the repository content and the working directory can be specified irrespective of the project location on the hosting computer. While this is most evident when working interactively in RStudio, the **R package {here}** allows to also specify file paths in relative to the `.Rproj` file. This enables the written code to run consistently, whether executed interactively in RStudio, statically from the command line using `RScript`, or integrated in an `Rmarkdown` (or `quarto`) document located in a subfolder.

git Integration. As further convenient feature, the version control of your project can be managed within RStudio if the RStudio project and git repository are in fact the same folder (which is recommended). In such cases, a dedicated `git` tab will become available within RStudio, and management of the version control can be conducted with minimal interruption of the coding workflow (note that this is also always possible when using `VS code` or `Positorn` as IDE).

Working Environment. Finally, the **R package {renv}** provides a way to capture and

lock the exact composition of R packages used within an RStudio project. This can guard your code from breaking changes in package updates and allows to quickly set up the same environment with the exact R package versions used on different machines and to re-run your R code in similar conditions as on the original computer.

Recap

The main aim of the proposed structure is to enable you to pluck your project from your laptop, **put it into a different computer and directly run your code** there. To enable this, the **project should be self-contained** and house all needed data and code.

A good way to enable easy orientation, as well as disassembling and re-assembling your project is **structure it by input (data), instructions (code) and output (results)**.

If you have data that is required as input in multiple projects, you can **use symbolic links** to avoid unnecessary copies of these files. When working in R, RStudio projects **unlock convenient features** that can be used to write more reproducible R code, such and provided by the R packages **{here}** and **{renv}**.

REFERENCES AND FURTHER READING

- Alston, J.M. and Rick, J.A. (2021). A Beginner's Guide to Conducting Reproducible Research. *The Bulletin of the Ecological Society of America*, 102(2):e01801. DOI: [10.1002/bes2.1801](https://doi.org/10.1002/bes2.1801).
- Bryan, J. *et al.* (2024). What They Forgot to Teach You About R. <https://rstats.wtf>, accessed 2024-03-12.
- Figueiredo, L., Scherer, C. and Cabral, J.S. (2022). A simple kit to use computational notebooks for more openness, reproducibility, and productivity in research. *PLOS Computational Biology*, 18(9):e1010356. DOI: [10.1371/journal.pcbi.1010356](https://doi.org/10.1371/journal.pcbi.1010356).
- Noble, W.S. (2009). A quick guide to organizing computational biology projects. *PLOS Computational Biology*, 5(7):e1000424.
- Wilson, G. *et al.* (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510.